Pre-contenido — Operaciones con Cadenas y Aritmética

Arquitectura de Computadores · Unidad 8

Estudiante: González López

Parte 1 — Instrucciones de cadena y flag DF

Pregunta 1 — Predicción de consecuencias

Al comenzar `REP MOVSB`, CX vale 60000 y SI y DI apuntan al siguiente byte de origen y destino. Cada iteración lee un byte, lo escribe y, con DF=0, incrementa SI y DI y decrementa CX. Si IRQ0 se acepta entre iteraciones, el procesador conserva el estado necesario para reanudar la repetición; en la arquitectura x86 moderna, el marco de interrupción puede reflejar la continuación de la instrucción REP, de modo que el retorno vuelve a ejecutar el prefijo con los índices y el contador ya actualizados. Por ejemplo, si se han copiado 12000 bytes, quedan CX=48000, SI=origen+12000 y DI=destino+12000. El manejador debe preservar los registros que modifique según la convención del sistema y retornar con IRET. Tras atender la interrupción, la copia continúa donde quedó: no se vuelven a copiar los bytes completados ni se pierden los restantes. La interrupción por sí sola no corrompe el bloque; sí habría riesgos si el manejador modifica SI, DI, CX, DS, ES o los datos sin restaurarlos, o si otra tarea accede concurrentemente al búfer.

En un bucle explícito, si la interrupción llega justo después de `MOVSB` y antes de `LOOP`, SI y DI ya avanzaron un byte pero CX aún no disminuyó; al retornar, `LOOP` lo decrementa una sola vez y continúa. Si ocurre después del salto, la siguiente iteración aún no se ha copiado. Por tanto, el bucle tampoco duplica datos si el manejador restaura los registros que usa. REP concentra el movimiento y la repetición en una instrucción arquitectónicamente reanudable, mientras que el bucle expone más puntos de estado entre operaciones. Además, `MOVSB` solo accede a memoria; si “puerto” significa un puerto de E/S x86, la copia necesitaría `INSB` u operaciones de E/S, no `MOVSB` (Intel, 2024; AMD, 2024).

Pregunta 2 — Reflexión crítica

La elección de ES:DI como destino fijo simplifica la interpretación de las instrucciones de cadena originales: el opcode identifica la operación y el procesador conoce implícitamente qué índices y segmentos intervienen. Un prefijo de override permite cambiar el segmento de la fuente DS:SI; para el destino no existe un override equivalente. Es razonable entenderlo como parte de una codificación compacta y de una decisión de diseño del 8086, aunque no como una imposición universal a toda la ISA: codificar explícitamente dos segmentos habría requerido más bits u otra instrucción. La arquitectura mantuvo la compatibilidad, por lo que el inconveniente persiste incluso cuando los registros de índice se amplían a ESI/EDI en código de 32 bits.

En un kernel, copiar desde una región basada en FS o GS a un destino cuyo selector no está cargado en ES requiere cambiar ES temporalmente, copiar por partes, usar una instrucción alternativa o recurrir a una rutina especializada. Cambiar un selector puede ser costoso y peligroso si no se conserva el estado; también puede interferir con convenciones del sistema operativo. En modo protegido de 32 bits, ESI, EDI y ECX son los índices implícitos; el prefijo de tamaño de dirección 67h permite alternar temporalmente entre índices de 16 y 32 bits. En modo largo de 64 bits, la segmentación está muy limitada para direcciones ordinarias, aunque FS y GS conservan usos especiales y el destino implícito sigue siendo relevante. La restricción es hoy principalmente un legado de compatibilidad, no una optimización que deba imponerse a nuevos diseños. En una rutina robusta se documenta el efecto sobre ES y se preserva el registro cuando el ABI o el llamador lo requieren (Intel, 2024; AMD, 2024).

Pregunta 3 — Decisión de implementación

Para un kernel de 32 bits conviene seleccionar la estrategia según CPU, tamaño y alineación, en lugar de suponer que MOVSD siempre es más rápida. Sin ERMSB, REP MOVSD puede procesar cuatro bytes por iteración y suele ser una opción favorable para bloques grandes, especialmente si origen y destino están alineados. Sin embargo, la ventaja depende de microarquitectura, caché y alineación; un bloque pequeño puede no compensar el cálculo de palabras y el control del remanente. Se debe medir en las plataformas objetivo y respetar los casos de solapamiento: una copia hacia atrás requiere invertir el sentido y ajustar los índices.

Con ERMSB, la CPU puede acelerar `REP MOVSB` mediante mecanismos optimizados, de modo que la granularidad de byte no implica necesariamente menor rendimiento. MOVSB simplifica el caso de cualquier longitud y evita leer o escribir fuera del bloque para completar una palabra. MOVSD requiere procesar `n / 4` elementos y después `n mod 4` bytes; si los punteros no están alineados, el código debe considerar también una fase de alineación o aceptar el costo correspondiente. La recomendación es usar REP MOVSB como ruta simple y correcta, y escoger REP MOVSD o una estrategia híbrida solo para procesadores sin ERMSB cuando los benchmarks justifican la complejidad. Así se mantiene una ruta de remanente clara y se evita optimizar a partir de una regla histórica que ya no describe todas las CPU (Intel, 2024; Intel, 2023).

Parte 2 — Aritmética BCD y decimales empaquetados

Pregunta 1 — Análisis comparativo

La suma manual con `ADC` y `DAA` escala a tantos dígitos como permita el espacio reservado: cada byte almacena dos dígitos BCD y el código procesa los bytes de menor a mayor peso, transfiriendo el acarreo decimal mediante CF. Para seis dígitos se ejecutan tres sumas y tres ajustes, además de cargas, almacenamientos, índices y control de bucle; la rutina debe cuidar el primer byte, que no recibe acarreo previo. Su principal ventaja es que opera directamente sobre enteros decimales y no necesita convertir todo el número a otro formato. Su límite práctico es la implementación: el tamaño solo queda limitado por memoria y por la lógica de control escrita por el programador.

`FBLD` carga hasta 18 dígitos BCD empaquetados desde 10 bytes a ST(0), y `FBSTP` almacena un entero BCD de 18 dígitos y desapila el valor. No son por sí solas una suma: todavía se necesitan instrucciones x87 para sumar, comprobar rango y decidir cómo redondear al almacenar. Para un valor de seis dígitos, ambas instrucciones pueden mover el operando completo de una vez, pero la secuencia completa requiere cargar los dos operandos, operar, almacenar y controlar el estado. x87 resulta útil si siguen varias operaciones numéricas sobre valores ya cargados; para una suma aislada pequeña, una rutina entera puede ser más explícita y evita conversión a punto flotante extendido. El programador también debe considerar excepciones y redondeo de `FBSTP`. En consecuencia, no basta comparar el conteo de opcodes: importan el costo de conversión y el tipo de cálculo (Intel, 2024; Intel, 2023).

Pregunta 2 — Síntesis

Considérese la suma BCD empaquetada de 2587 + 1468, representada en bytes de menor peso primero: 87h, 25h y 00h para 2587; 68h, 14h y 00h para 1468. En el byte de unidades y decenas, `87h + 68h` produce E Fh en la suma binaria; después de `DAA`, AL queda 55h y CF=1, que representa el acarreo decimal de 100 hacia los dígitos siguientes. La siguiente operación debe ser `ADC 25h,14h`: suma los bytes y añade ese CF, y `DAA` deja 40h con CF=0. El byte más significativo suma 00h+00h y queda 00h. El resultado en bytes es 00h,40h,55h, es decir, 4055 decimal.

CF es el canal entre iteraciones: `DAA` produce un acarreo decimal, y `ADC` lo incorpora al siguiente byte antes del siguiente ajuste. Sustituir `ADC` por `ADD` en el byte intermedio ignora el acarreo de unidades/decenas. En el ejemplo se obtendría 39h en vez de 40h, por lo que el resultado sería 3955, aunque cada byte parezca BCD válido. El error no necesariamente provoca una excepción ni una señal visible; es un fallo lógico que se manifiesta solo cuando la posición anterior genera acarreo. La misma disciplina se aplica a la resta: `SBB` consume el préstamo del byte previo y `DAS` ajusta cada resultado antes de propagarlo (Intel, 2024; IBM, n.d.).

Pregunta 3 — Investigación de casos

El tipo COBOL `COMP-3` o `PACKED-DECIMAL` describe una representación decimal empaquetada que el programador conserva en el nivel del lenguaje; no obliga al compilador a emitir las instrucciones x86 `DAA`, `DAS` o `AAA`. En x86-64, las instrucciones de ajuste ASCII/decimal citadas no son válidas en modo largo y generan #UD, pero eso no elimina el tipo COBOL. El compilador puede implementar sus operaciones con secuencias de instrucciones enteras, rutinas de biblioteca o una representación intermedia diferente, conservando el contrato decimal observable.

Un caso técnico verificable es IBM Enterprise COBOL for Linux on x86, que ofrece compatibilidad COBOL y tipos decimales sobre Linux x86-64; su documentación del compilador describe las opciones de plataforma, tipos numéricos y generación de código sin requerir el opcode `DAA`. En el otro extremo, IBM z/Architecture mantiene instrucciones decimales decodificadas por hardware, una opción natural para cargas COBOL empresariales que ya residen en mainframe. Por tanto, la transición no fue una única migración universal: las aplicaciones que pasaron a x86-64 pudieron conservar el modelo decimal mediante implementación del compilador/runtime; organizaciones con dependencias de servicio y volúmenes mainframe también mantuvieron cargas en z/OS. La Intel Decimal Floating-Point Math Library es una posibilidad para cálculos de punto flotante decimal, pero no es requisito general para ejecutar `COMP-3`, ni reemplaza automáticamente toda semántica COBOL de decimal fijo. La fuente del compilador de IBM permite documentar el caso de plataforma x86, mientras que el manual de Intel confirma la invalidez del opcode en 64 bits (Intel, 2024; IBM, n.d.).

Apéndice — Ejemplos y verificación de los algoritmos

Cadenas: SI, DI, CX y DF

En cada operación, CLD deja DF=0, por lo que los índices avanzan. REP MOVSB decrementa CX por byte; al copiar 13 bytes, SI y DI avanzan 13 posiciones y CX termina en cero.

```
BITS 16
mov ax, ds
mov es, ax
mov si, origen
mov di, destino
mov cx, longitud
cld
rep movsb
```

La instrucción SCASB compara AL con el byte de ES:DI. REPNE se detiene si encuentra el valor (ZF=1) o si CX llega a cero; DI queda después del último byte examinado. En CMPSB se comparan DS:SI y ES:DI; REPE continúa mientras sean iguales y quede contador.

```
mov di, cadena
mov al, 'd'
mov cx, longitud
cld
repne scasb
jne no_encontrado       ; ZF=0: terminó CX sin coincidencia

mov si, cadena_a
mov di, cadena_b
mov cx, longitud_conocida
cld
repe cmpsb
jne diferentes          ; ZF=0: primer byte distinto
```

El override cambia solo la fuente: `CS:SI` es válido, pero el destino de MOVS permanece `ES:DI`. La variante de 32 bits usa ESI, EDI y ECX en un segmento de código de 32 bits:

```
cs rep movsb            ; fuente CS:SI, destino ES:DI

BITS 32
cld
mov esi, origen
mov edi, destino
mov ecx, dwords
rep movsd               ; copia ECX elementos de cuatro bytes
```

Ajustes BCD y acarreo de varios bytes

DAA usa el nibble bajo de AL y AF después de ADD: si `(AL AND 0Fh)>9` o AF=1, suma 6 y fija AF; de lo contrario limpia AF. Luego evalúa el AL y CF **anteriores a DAA**: si el AL original supera 99h o CF previo estaba activo, suma 60h y fija CF; de lo contrario limpia CF. DAS sigue la corrección por resta con las mismas pruebas sobre el nibble bajo/AF y el AL/CF previos, restando 6 y 60h, respectivamente.

- **DAA:** 47h + 38h = 7Fh; el nibble bajo F supera 9, así que se suma 06h y AL pasa a 85h; el AL previo 7Fh no supera 99h y CF previo es cero, por lo que no se suma 60h. Resultado BCD: 85h, AF=1 y CF=0.

- **DAS:** 73h − 28h = 4Bh; la resta del nibble bajo produjo B y AF=1, así que se resta 06h y AL pasa a 45h. El AL previo no supera 99h y no hubo préstamo de byte, por lo que no se resta 60h. Resultado BCD: 45h, CF=0.

Para sumar tres bytes BCD, se recorren de menor a mayor peso. La primera columna produce `87h + 68h = EFh`; el nibble bajo F no genera acarreo binario, así que AF=0 antes de DAA. DAA ajusta el resultado a 55h, establece AF=1 y deja CF=1. La columna siguiente usa `ADC`: `25h + 14h + CF(1) = 3Ah`; el nibble bajo A tampoco desborda cuatro bits, así que AF=0 antes de DAA. El ajuste deja 40h, AF=1 y CF=0. La columna superior usa `ADC` con CF=0: `00h + 00h = 00h`, AF=0 y CF=0. Los bytes almacenados de mayor a menor peso son 00h, 40h, 55h: el resultado es 4055. Si la columna intermedia usara ADD, se perdería el acarreo y produciría 3955.

AAA ajusta una suma desempaquetada cuando el nibble bajo de AL excede 9 o AF=1: suma 0106h a AX, fija AF y CF, y luego enmascara AL a 0Fh. AAS resta 0106h bajo la condición análoga y limpia el nibble alto de AL. Para `07h+08h=0Fh`, AAA produce AH:AL=01:05. AAM con su base predeterminada 10 divide AL en decenas (AH) y unidades (AL); `7×8=56` se vuelve 05:06. AAD calcula `AL=AH×10+AL` y deja AH=0, preparando el entero para DIV: 05:06 pasa a AL=56 decimal. Las seis instrucciones DAA, DAS, AAA, AAS, AAM y AAD son inválidas en modo largo x86-64 (#UD); FBLD/FBSTP son instrucciones x87 distintas para cargar/almacenar BCD empaquetado extendido. IEEE 754 decimal y DPD conservan el objetivo de representar decimales, pero no son equivalentes a ejecutar esos opcodes (Intel, 2024; IBM, n.d.).

Referencias

AMD. (2024). *AMD64 Architecture Programmer's Manual, Volume 3: General-Purpose and System Instructions*. https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/24594.pdf

IBM. (n.d.). *IBM COBOL for Linux on x86 documentation*. https://www.ibm.com/docs/en/cobol-linux-x86

IBM. (n.d.). *Enterprise COBOL for z/OS: Packed-decimal data*. https://www.ibm.com/docs/en/cobol-zos/6.4?topic=concepts-packed-decimal-data

Intel. (2023). *Intel 64 and IA-32 Architectures Optimization Reference Manual*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

Intel. (2024). *Intel 64 and IA-32 Architectures Software Developer's Manual* (Vols. 2–3). https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html